

Arquitectura y Diseño de Sistemas 2026

Pipeline de Datos

Documentación del flujo de procesamiento de datos — Proyecto Polizing

Entregable 4 — Práctica de Arquitectura y Diseño de Sistemas 2026

Este documento describe los pipelines de datos del sistema Polizing, siguiendo los lineamientos de la cátedra. Se documentan dos pipelines: (1) el scraper de noticias desde una fuente RSS externa hasta su presentación en el frontend del dashboard interno, y (2) la sincronización diaria de datos desde los Web Services SOAP de Berkley hasta la base de datos del sistema.

1. Identificación de Pipelines

El sistema Polizing cuenta con dos pipelines de datos identificados:

1. Scraping y Sincronización de Noticias — ETL bajo demanda con caché que extrae titulares del feed RSS de Asegurando Digital.
2. Sincronización Diaria de Datos con Berkley — ETL batch programado que consulta el WS de Novedades de Berkley, descarga archivos GD y los persiste en la base de datos.

A continuación se analiza el sistema completo para confirmar el inventario de flujos de datos.

1.1. Inventario de posibles flujos de datos

Flujo	¿Es pipeline de datos?	Justificación
Scraping de noticias (asegurandodigital.com.ar)	Sí	Extrae datos externos, los transforma y los persiste en DB. Pipeline ETL clásico documentado en la Sección 2.
Sincronización Berkley (WS SOAP apwsnovedades + archivos GD)	Sí	Extrae datos de una API externa (SOAP), descarga archivos de ancho fijo, los parsea, transforma y persiste en DB. Pipeline ETL batch documentado en la Sección 3.
Consulta de cliente por teléfono (chatbot → API)	No	Llamada síncrona REST sin transformación ni persistencia intermedia. No hay pipeline de datos.
Alta de solicitud de póliza (chatbot → API)	No	Operación CRUD simple, no hay transformación batch ni streaming.
Autenticación de usuarios (NextAuth / JWT)	No	Flujo de autenticación estándar, sin procesamiento de datos.
Carga de documentos (subida de archivos)	No	Persistencia directa sin transformación, operación CRUD.

2. Pipeline: Scraping y Sincronización de Noticias

Pipeline ETL que extrae titulares del feed RSS de Asegurando Digital (fuente externa de noticias del ámbito asegurador), los transforma (parseo, limpieza, validación) y los persiste en la base de datos PostgreSQL de Supabase para su consumo por el dashboard interno de Polizing.

Tipo de procesamiento: ETL (Extract-Transform-Load) con caché de resultados.

PASO 1 — ORIGEN DE LOS DATOS

¿**Qué datos**: Items del feed RSS de noticias. Cada item contiene: título, URL del artículo, fecha de publicación (RFC 2822), categoría, descripción en HTML, y contenido HTML embebido con imágenes.

¿**Desde dónde**: Endpoint público RSS: <https://asegurandodigital.com.ar/feed/>. El sitio expone un feed XML estándar compatible con WordPress (robots.txt no bloquea /feed/).

¿**Con qué frecuencia llegan**: No llegan automáticamente. El pipeline se ejecuta on-demand con caché de 1 hora (ver Paso 2).

Volumen estimado: El feed devuelve ~20 items por request. Se persisten 12 registros por ejecución (~4 KB total). Escala lineal si se agregan más fuentes.

PASO 2 — DISPARADOR / TRIGGER

¿**Cuándo se ejecuta**: Event-driven con throttle. Se dispara cuando un usuario visita la ruta /noticias del dashboard y la caché de Next.js está expirada. El perfil de caché 'hours' establece revalidate de 1 hora, por lo que el scraping real ocurre como máximo una vez por hora sin importar cuántos usuarios accedan simultáneamente.

¿**Quién lo dispara**: El server component NoticiasPage (Next.js App Router) llama a getNoticias() en lib/data/noticias.ts. Si la caché asociada al tag 'noticias' está vencida, se ejecuta refreshSnapshot(). Adicionalmente, el usuario puede disparar una ejecución forzada mediante el botón 'Actualizar' que invoca un Server Action (refreshNoticias → updateTag('noticias')).

PASO 3 — PROCESAMIENTO

¿**Qué componente lo ejecuta**: Next.js 16 runtime (Node.js). El código corre en el servidor, dentro del proceso de la aplicación web. No hay workers ni contenedores separados. Tecnologías: fetch (HTTP), cheerio (parseo XML), Zod (validación), Prisma ORM (persistencia).

Pasos del procesamiento:

#	Paso	Descripción	Detalle técnico
1	EXTRACT (Obtener feed)	Solicitud HTTP GET al feed RSS.	fetch() con timeout de 10s, User-Agent identificable ('Polizing-Interno-Bot/1.0'), cache: 'no-store'. Se rechaza si status ≠ 200.
2	TRANSFORM — Parseo (Extraer items)	Lectura del XML con cheerio en modo xmlMode.	Se itera cada <item> extrayendo <title>, <link>, <pubDate>, <category>, <description>, <content:encoded>. Se deduplica por URL.

3	TRANSFORM — Limpieza	Strip HTML del resumen, extraer primera imagen.	Se eliminan tags HTML, se decodifican entidades (&, <, ’, etc.), se trunca resumen a 280 caracteres.
4	TRANSFORM — Validación	Esquema Zod valida cada item.	NoticiaScrapeadaSchema exige: url_origen (URL válida), titulo (3-500 chars), resumen, imagen_url, publicada_en (Date), categoria. Items inválidos se descartan silenciosamente.
5	LOAD (Persistir en DB)	Transacción atómica: deleteMany + createMany.	Prisma.\$transaction elimina todos los registros existentes e inserta los nuevos. El snapshot anterior se reemplaza completamente.

Manejo de errores:

- **Error de red/parseo:** Si el fetch al feed falla (timeout, red, status no-200) o el parseo arroja excepción, el catch en refreshSnapshot() loguea el error y retorna sin modificar la tabla.
- **Scrape vacío:** Si el scrape devuelve 0 items (feed vacío o validación Zod descarta todo), la tabla no se modifica.
- **Preservación:** deleteMany solo se ejecuta si scraped.length > 0, preservando el último snapshot exitoso.
- **Estado vacío:** Si la DB está vacía y el primer scrape falla, la página muestra <NoticiasEmptyState> con mensaje amigable.
- **Logging:** "[noticias] scrape falló, sirviendo último snapshot de DB" aparece en consola del servidor.

PASO 4 — SALIDA / DESTINO

¿**Qué se produce:** Un snapshot de 12 registros de noticias limpios, tipados y validados. Cada registro contiene: id, url_origen, titulo, resumen, imagen_url, publicada_en, categoria, fuente ('Asegurando Digital'), scrapeada_en.

¿**Dónde se guarda:** Tabla noticias_scrapeadas en Supabase PostgreSQL (esquema public). La tabla tiene 9 columnas, índices descendentes en publicada_en y scrapeada_en, y unique constraint en url_origen.

¿**Quién consume el resultado:** El frontend del dashboard interno de Polizing. La función getNoticias() lee los registros y los devuelve tipados como NoticiaListItem[] al componente NoticiasPage, que renderiza NoticiasGrid → NoticiaCard. Adicionalmente, la API caché de Next.js 16 mantiene los datos en memoria para respuestas instantáneas entre re-scrapes.

PASO 5 — DECISIÓN ARQUITECTÓNICA

¿**Por qué este enfoque:** Se eligió un pipeline ETL clásico con caché de resultados porque: (1) la transformación en memoria antes de DB asegura que solo datos limpios y validados lleguen a la tabla — no se persiste XML crudo; (2) el frontend lee filas tipadas sin lógica de parseo; (3) la caché de Next.js evita scrapes repetidos contra el feed RSS, reduciendo la carga sobre la fuente externa; (4) el feed RSS es más estable que el HTML del sitio (no cambia con rediseños) y está permitido por robots.txt.

Trade-offs aceptados: Latencia: los datos no están en tiempo real. Pueden tener hasta 1 hora de desfase debido a la caché (perfil 'hours'). Consistencia: el snapshot se reemplaza completamente, no hay merge incremental. Si una noticia se elimina de la fuente, desaparece

del dashboard en el próximo scrape. Dependencia externa: si el feed RSS está caído, el pipeline no puede actualizar datos (aunque el último snapshot persiste en DB). Escalabilidad: el deleteMany + createMany no escala a miles de registros, pero es aceptable para el volumen actual (~12 registros cada hora).

3. Pipeline: Sincronización Diaria de Datos con Berkley

Pipeline ETL batch que consulta diariamente el Web Service SOAP de Novedades (apwsnovedades) de Berkley, descarga los archivos GD modificados desde una fecha dada, los parsea (formato de ancho fijo), los transforma y los persiste en la base de datos PostgreSQL de Supabase. Este pipeline mantiene actualizados los datos de pólizas, asegurados y movimientos del sistema Polizing contra la información registrada en la aseguradora Berkley.

Tipo de procesamiento: ETL (Extract-Transform-Load) batch con ejecución programada (cron).

PASO 1 — ORIGEN DE LOS DATOS

¿**Qué datos**: Dos niveles de datos: (a) metadatos de archivos desde el WS SOAP — nombre de archivo, fecha de última modificación, link de descarga; (b) archivos GD de ancho fijo (fixed-width) en formato ANSI/Windows-1252. Los archivos principales incluyen: Polizas2.txt (pólizas y datos principales), asegur.txt (asegurados/clientes), movimi.txt (movimientos contables), cdp.txt (cuotas y vencimientos), pagos.txt (pagos recibidos), más tablas maestras (ramas, cobert, monemi, tipveh, vehicu2, etc.).

¿**Desde dónde**: WS SOAP:

<https://ws.berkley.com.ar/BIWebWS/com.binet17.webservices.apwsnovedades> (producción). Descarga de archivos: links HTTPS devueltos por el WS. Las credenciales de acceso son provistas por Berkley (Usuario + Password específico para Novedades).

¿**Con qué frecuencia llegan**: No llegan automáticamente. El pipeline se ejecuta 1 vez por día (batch programado). Los datos reflejan cambios ocurridos en Berkley desde la última ejecución.

Volumen estimado: ~25 archivos disponibles por corrida, de los cuales se descargan y procesan entre 5 y 10 archivos relevantes. Volumen total estimado: 500 KB – 2 MB por ejecución. Cada archivo contiene entre decenas y cientos de registros (p. ej. Polizas2.txt puede contener cientos de pólizas activas).

PASO 2 — DISPARADOR / TRIGGER

¿**Cuándo se ejecuta**: Batch programado (cron). Todos los días a las 04:00 AM (huso horario de Argentina), horario seleccionado para minimizar impacto: bajo tráfico en el dashboard, baja probabilidad de contención con usuarios reales, y ventana suficiente antes del inicio de la jornada laboral.

¿**Quién lo dispara**: Vercel Cron Jobs. Definido mediante la propiedad crons en vercel.json con expresión CRON (0 7 * * * en UTC, equivalente a 4 AM ART). Vercel ejecuta una función serverless (API Route, p. ej. GET /api/cron/berkley-sync) que inicia todo el flujo. El scheduler de Vercel garantiza una ejecución por día, con reintentos automáticos en caso de falla de infraestructura.

PASO 3 — PROCESAMIENTO

¿**Qué componente lo ejecuta**: Next.js 16 runtime (Node.js). El código corre como serverless function dentro del mismo proyecto en Vercel, invocada por el cron job. Tecnologías: fetch / axios (HTTP para SOAP y descarga de archivos), construcción manual de envelope XML SOAP, parser de ancho fijo (implementación con slice() y strip()), Zod (validación), Prisma ORM (persistencia upsert).

Pasos del procesamiento:

#	Paso	Descripción	Detalle técnico
1	EXTRACT — Consultar WS Novedades	Invocar apwsnovedades vía SOAP con FechaDesde = ultimaCorrida.	Se envía POST con envelope XML a endpoint SOAP. Header Content-Type: text/xml; charset=utf-8. El response contiene ErrorCodigo (0 = ok) y lista de archivos disponibles con nombre, fecha y link de descarga.
2	EXTRACT — Descargar archivos	Por cada archivo devuelto y relevante, descargar el contenido mediante GET HTTPS.	Se usa fetch() con autenticación Basic Auth (mismas credenciales del WS). Se verifica HTTP status 200. Si el link expiró, se loguea y se omite ese archivo.
3	TRANSFORM — Parseo fixed-width	Leer cada archivo línea por línea y extraer campos según el layout específico (posición desde, longitud).	Cada línea se procesa con line[desde-1 : desde-1+longitud].strip(). Encoding latin1 (Windows-1252). Fechas de AAAAMMDD a Date. Números con parseInt/parseFloat.
4	TRANSFORM — Validación	Validar cada registro parseado con esquema Zod.	Cada archivo tiene su esquema (Polizas2Schema, AsegurSchema). Registros inválidos se descartan y loguean. Se verifica integridad referencial mínima.
5	TRANSFORM — Normalización	Normalizar tipos de datos y enriquecer con tablas maestras.	Convertir monedas según monemi, resolver descripciones de ramas, clasificar tipos de vehículo. Estandarizar formato de CUIT, teléfonos y direcciones.
6	LOAD — Persistir en DB	Insertar o actualizar registros mediante upsert por clave natural.	prisma.\$transaction con upsert (rama + nro_póliza + suplemento). Primera ejecución: createMany. Tablas maestras: deleteMany + createMany.

Manejo de errores:

- **Error de red SOAP:** Si la llamada al WS falla (timeout, DNS, TLS), la ejecución se aborta y se loguea. No se avanza ultimaCorrida.
- **Error de descarga:** Si la descarga de un archivo individual falla (HTTP no-200, link expirado), se omite ese archivo pero se continúa con el resto.
- **Error de parseo:** Si un archivo tiene formato inesperado, se descarta el archivo completo y se alerta al administrador.
- **Error de persistencia:** Si falla la transacción de BD, Prisma hace rollback automático. No se persisten datos parciales.
- **Conservación de ultimaCorrida:** Siguiendo patrón Berkley (§2.8), ultimaCorrida solo avanza si todo el pipeline se completa exitosamente.
- **Logging:** Cada paso registra: archivos encontrados, descargados, parseados, registros válidos/inválidos y persistidos.

PASO 4 — SALIDA / DESTINO

¿**Qué se produce**: Registros de pólizas, asegurados, movimientos contables, cuotas y pagos, normalizados, tipados y validados. Cada registro mantiene claves naturales (rama, nro_póliza, suplemento) para upserts idempotentes.

¿**Dónde se guarda**: Tablas en Supabase PostgreSQL (esquema public): polizas (desde Polizas2.txt), asegurados (desde asegur.txt), movimientos (desde movimi.txt), cuotas (desde cdp.txt), pagos (desde pagos.txt), y tablas maestras. Cada tabla con índices por clave natural y constraint de unicidad.

¿**Quién consume el resultado**: Frontend del dashboard (visualización de pólizas, clientes, cuenta corriente), API REST del sistema, y potencialmente otros subsistemas.

PASO 5 — DECISIÓN ARQUITECTÓNICA

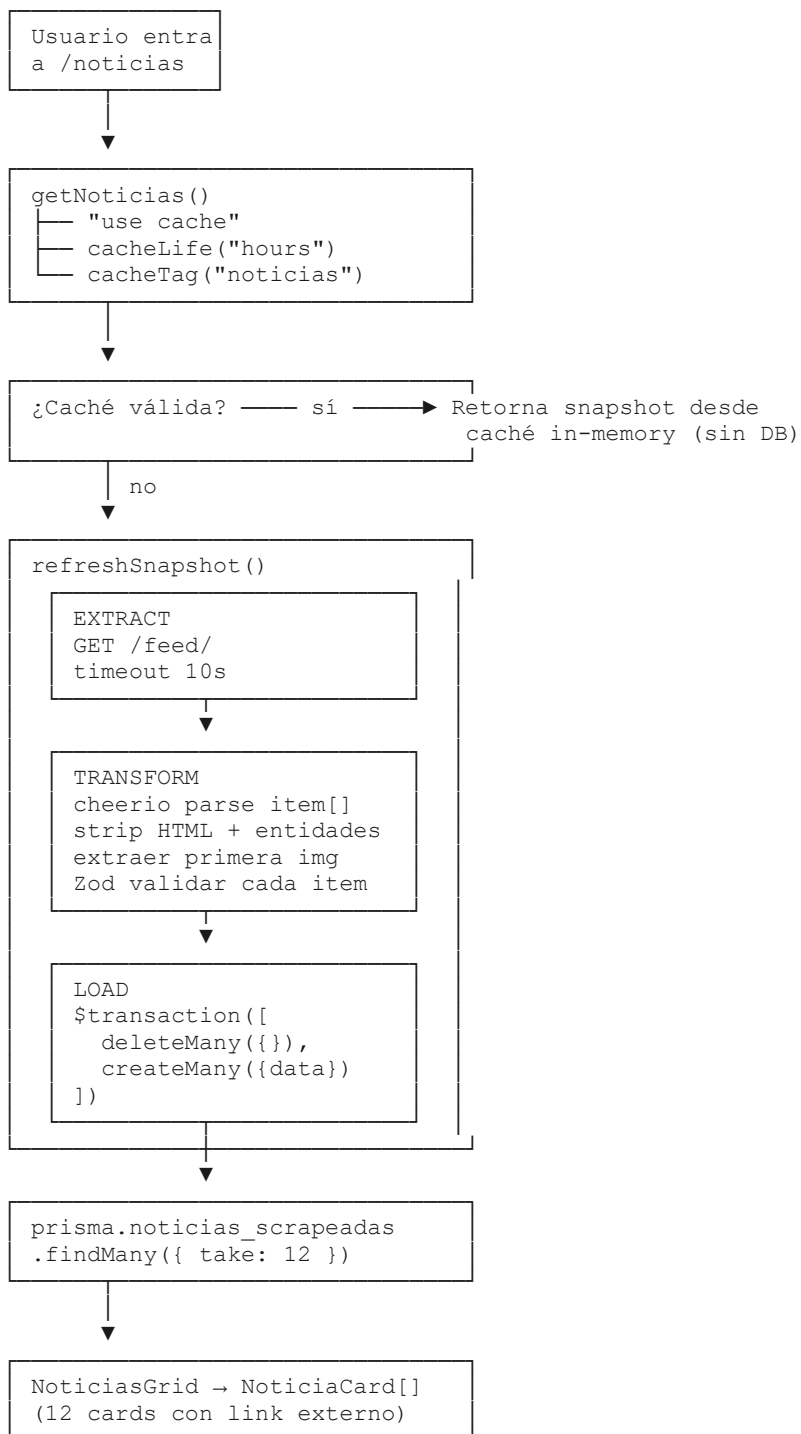
¿**Por qué este enfoque**: Se eligió pipeline ETL batch nocturno porque: (1) los datos no requieren sincronización en tiempo real — actualización diaria es suficiente; (2) el WS apwsnovedades está diseñado para sincronización diaria incremental (§6.1 manual Berkley); (3) el horario nocturno (4 AM) minimiza contención con tráfico real; (4) se aprovecha Vercel Cron Jobs sin workers separados; (5) el parseo fixed-width es estable porque los layouts de Berkley cambian con poca frecuencia.

Trade-offs aceptados: Latencia: hasta 24h de desfase. Ventana de ejecución: si falla, no hay reintento hasta el día siguiente. Complejidad: layouts fixed-width pueden romperse si Berkley los modifica sin aviso. Dependencia externa: si el WS está caído, no se actualizan datos. Autenticación: credenciales SOAP como secretos en Vercel.

4. Diagramas de Flujo

4.1. Pipeline de Scraping de Noticias

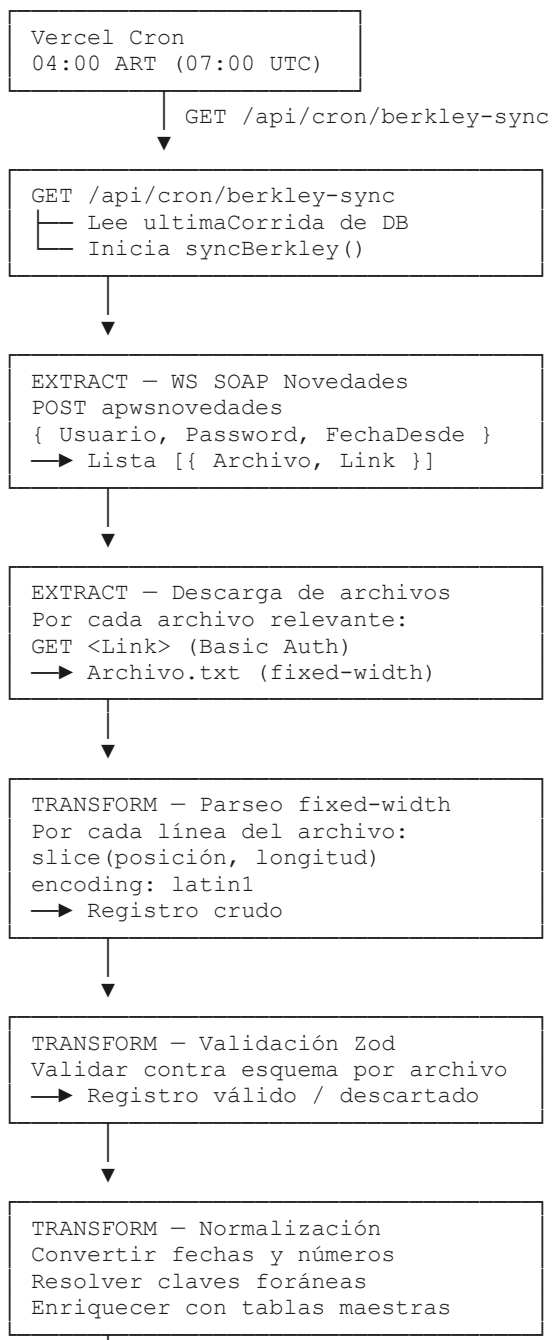
A continuación se describe el flujo completo del pipeline de noticias, desde la solicitud del usuario hasta la renderización de los datos.

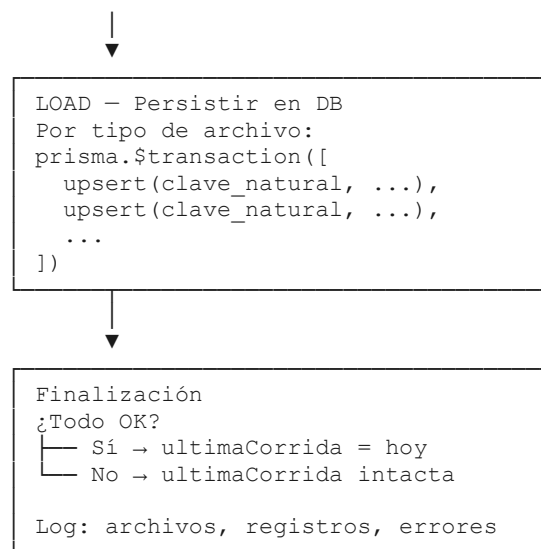


El diagrama muestra el flujo completo del pipeline, incluyendo el bypass de caché que evita ejecuciones redundantes del ETL.

4.2. Pipeline de Sincronización Diaria con Berkley

A continuación se describe el flujo completo del pipeline de sincronización Berkley, desde el disparo del cron hasta la persistencia en base de datos.





El diagrama muestra el flujo completo del pipeline, incluyendo la lógica de conservación de ultimaCorrida que evita pérdida de datos si la ejecución falla.